FIAP | Análise e Desenvolvimento de Sistemas

Desafio DevOps

Elos ESG API

Projeto Cidades ESG Inteligentes: CI/CD, containerização e deploy em staging e produção

Integrante: Gabriel Bueno Barba

RM: 572765

Turma: 2TDSOA

Repositório: https://github.com/heyblurryface/elos-esg-api

Data: outubro de 2026

1. O projeto

O Elos ESG é uma plataforma de diagnóstico e acompanhamento da maturidade ESG de empresas e fornecedores. O núcleo técnico usado neste desafio é a API de diagnóstico desenvolvida na fase anterior como Kaizen API, em C# com ASP.NET Core 8, Entity Framework Core e banco Oracle.

Na API, o consultor cadastra a empresa, abre uma avaliação e responde 32 perguntas distribuídas em 5 pilares (Governança, Segurança da Informação, Ética e Integridade, Políticas e Documentação, Terceiros e Conscientização). A cada resposta a própria API recalcula o score do pilar, classifica o nível de maturidade, gera a recomendação e atualiza o status da avaliação.

Para este desafio a aplicação foi adaptada para um cenário de produção real:

- imagem Docker multi-stage, rodando com usuário sem privilégios e com healthcheck;
- Docker Compose orquestrando API e Oracle com volume persistente, redes separadas e variáveis de ambiente;
- novo endpoint /health, que mostra o estado do banco, o ambiente e a versão publicada;
- pipeline de CI/CD no GitHub Actions com build, testes, publicação da imagem e deploy automatizado em staging e produção;
- dois ambientes isolados numa VM Linux na Azure, com aprovação manual antes de produção.

2. Pipeline de CI/CD

2.1 Ferramenta

Foi usado o GitHub Actions, porque o código já fica no GitHub, a imagem pode ser publicada no GitHub Container Registry (GHCR) com o próprio token do workflow e o recurso de Environments permite separar os secrets de cada ambiente e exigir aprovação antes de produção.

O pipeline está em dois arquivos:

Arquivo	Função
.github/workflows/ci-cd.yml	Pipeline principal. Dispara em push na main, em pull request e manualmente (Run workflow).
.github/workflows/deploy.yml	Workflow reutilizável de deploy. É chamado duas vezes, para staging e para produção, só mudando os parâmetros (ambiente, porta, imagem e versão).

2.2 Etapas

Etapa	O que acontece
1. Build e testes	Instala o .NET 8, faz restore e build em Release e roda os 26 testes xUnit (integração dos 7 controllers e do /health, e unitários da engine de diagnóstico). Gera o relatório .trx como artefato e um resumo na página da execução. Se um teste falhar, nada é publicado.
2. Imagem Docker	Valida o docker-compose.yml, faz o build da imagem com cache do Buildx e publica no GHCR com as tags sha-<commit> e latest. Em pull request só faz o build, sem publicar.
3. Deploy staging	Automático. Conecta na VM por SSH, gera o .env com os secrets do environment staging, copia o compose, baixa a imagem e executa docker compose up --wait. Depois

Etapa	O que acontece
	roda o smoke test de fora da VM.
4. Aprovação	O environment production tem required reviewer. O pipeline para e espera alguém clicar em Approve.
5. Deploy produção	Mesmo processo com os secrets de produção, na porta 8080, seguido de smoke test.

2.3 Lógica do pipeline

- **Uma imagem, dois ambientes:** a imagem marcada com o SHA do commit é construída uma vez e promovida de staging para produção sem recompilar. O que foi testado em staging é exatamente o que vai para produção.
- **Ambientes isolados:** na VM cada ambiente é um projeto do Docker Compose (elos-staging e elos-production), com containers, rede, volume do Oracle e senhas próprios.
- **Verificação real do deploy:** o smoke test espera o /health responder 200, confere se a versão publicada é a do commit, faz login com JWT e chama uma rota protegida (/api/empresas).
- **Segredos fora do código:** senhas do Oracle e chave JWT ficam nos GitHub Environments. O .env é gerado na hora do deploy, copiado com permissão 600 e o token do registry chega ao script remoto pela entrada padrão, sem aparecer na linha de comando.
- **Segurança do fluxo:** pull requests passam por build, testes e build da imagem, mas não publicam nem fazem deploy. Os deploys usam concurrency para não rodar dois ao mesmo tempo no mesmo ambiente.

2.4 Infraestrutura na Azure

Os dois ambientes rodam numa Virtual Machine Ubuntu 22.04 (Standard_B2s_v2, 2 vCPU e 8 GB) na região Chile Central, criada pelo script infra/azure-setup.sh no Azure Cloud Shell. O script instala o Docker via cloud-init, cria swap, libera no NSG as portas 8081 (staging) e 8080 (produção) e gera uma chave SSH exclusiva para o pipeline. O script infra/github-setup.sh configura as variables, os secrets e os environments do repositório pelo GitHub CLI.

Ambiente	Projeto Compose	Porta	ASPNETCORE_ENVIRONMENT	Deploy
Staging	elos-staging	8081	Staging	Automático após a imagem
Produção	elos-production	8080	Production	Após staging OK + aprovação

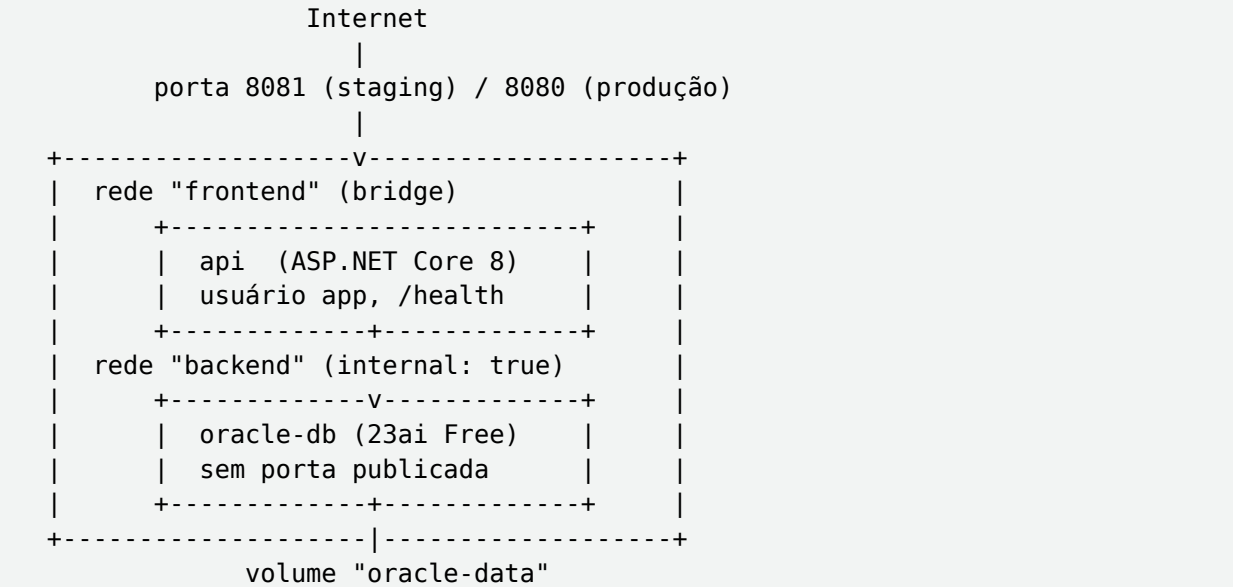
Endereços: staging em http://68.211.144.197:8081/swagger e produção em http://68.211.144.197:8080/swagger (disponíveis enquanto a VM estiver ligada).

Execução validada	Resultado
Execução	CI/CD - Elos ESG API #2, commit 2b0edc4
Build e testes	26 testes executados, 26 aprovados (39s)
Imagem	ghcr.io/heyblurryface/elos-esg-api:sha-2b0edc4 (47s)
Deploy staging	Sucesso em 1m43s, smoke test OK

Execução validada	Resultado
Deploy produção	Aprovado manualmente, sucesso em 1m16s, smoke test OK

3. Docker

3.1 Arquitetura



O banco fica só na rede interna backend, que não tem saída para a internet e não publica nenhuma porta no host. A API participa das duas redes: fala com o Oracle pela backend e é exposta pela frontend. Os dados do Oracle ficam no volume oracle-data e sobrevivem a restarts e novos deploys.

3.2 Imagem criada

Item	Valor
Nome	ghcr.io/heyblurryface/elos-esg-api:sha-2b0edc4 (e :latest)
Base final	mcr.microsoft.com/dotnet/aspnet:8.0 (só runtime, sem SDK)
Estágios	restore, build, test, publish, final
Usuário	app (não-root)
Porta	8080
Healthcheck	curl -fsS http://localhost:8080/health a cada 30s
Versão	build-arg APP_VERSION = SHA do commit, exposto no /health e nos labels OCI

Estratégias: os arquivos .csproj são copiados e restaurados antes do código para aproveitar o cache de camadas; o estágio test permite rodar os testes dentro do container (docker build --target test); nenhum segredo entra na imagem, connection string e chave JWT chegam por variável de ambiente; e o .dockerignore deixa de fora bin, obj, .git, documentação e arquivos .env.

3.3 docker-compose.yml

Recurso	Uso
Serviços	oracle-db e api
Volume	oracle-data em /opt/oracle/oradata
Redes	backend (interna) e frontend
Variáveis	Lidas do .env: ambiente, porta, imagem, versão, senhas e chave JWT. As obrigatórias usam ${VAR:?msg}.
Ordem de subida	depends_on com condition: service_healthy
Healthchecks	healthcheck.sh no Oracle e curl /health na API
Resiliência	restart: unless-stopped e rotação de logs

3.4 Comandos usados

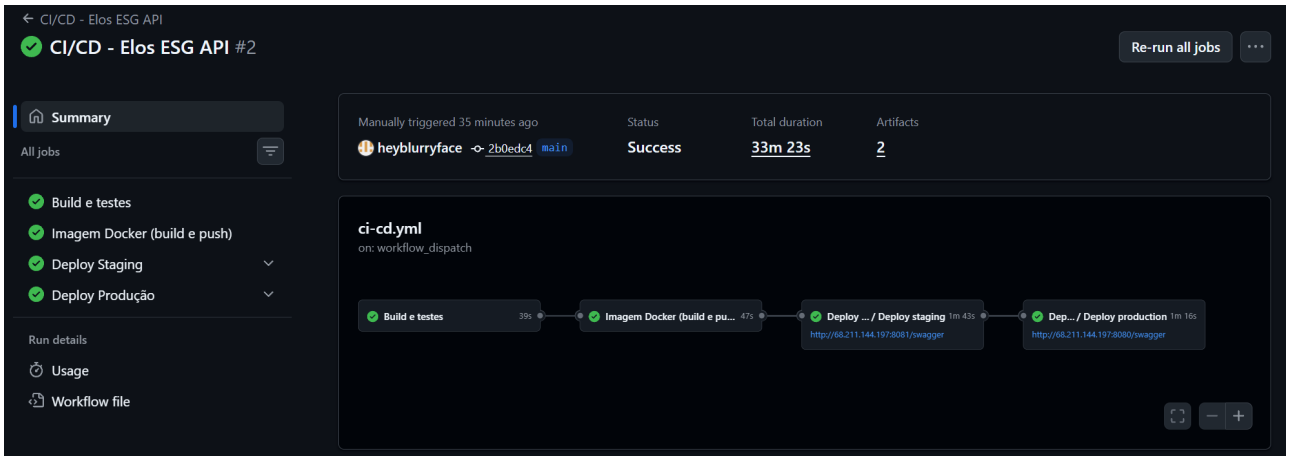
```
# local
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f api
./scripts/smoke-test.sh http://localhost:8080

# testes dentro do container
docker build --target test -t elos-esg-api:test .

# na VM (executado pelo pipeline)
docker compose -p elos-staging pull
docker compose -p elos-staging up -d --no-build --wait --wait-timeout 900
docker compose -p elos-production ps
```

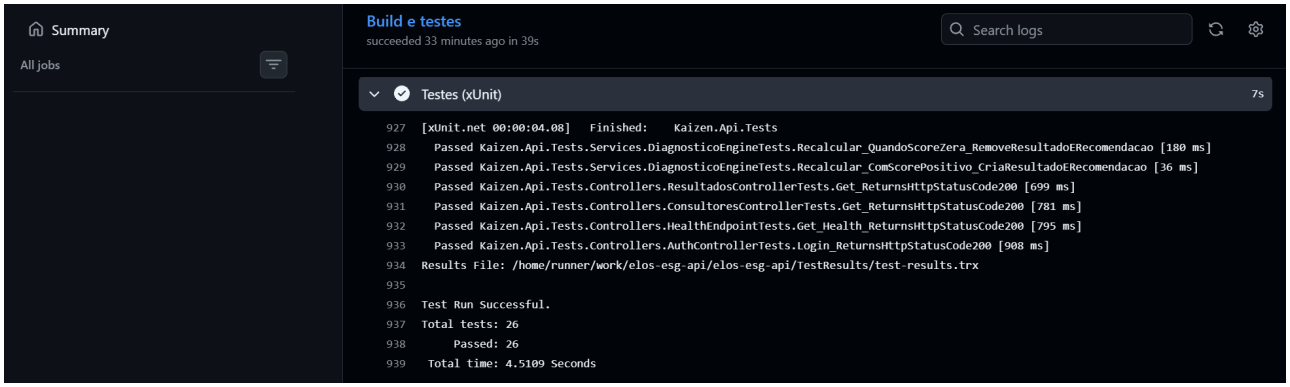
4. Evidências do pipeline

Figura 1. Execução completa do pipeline



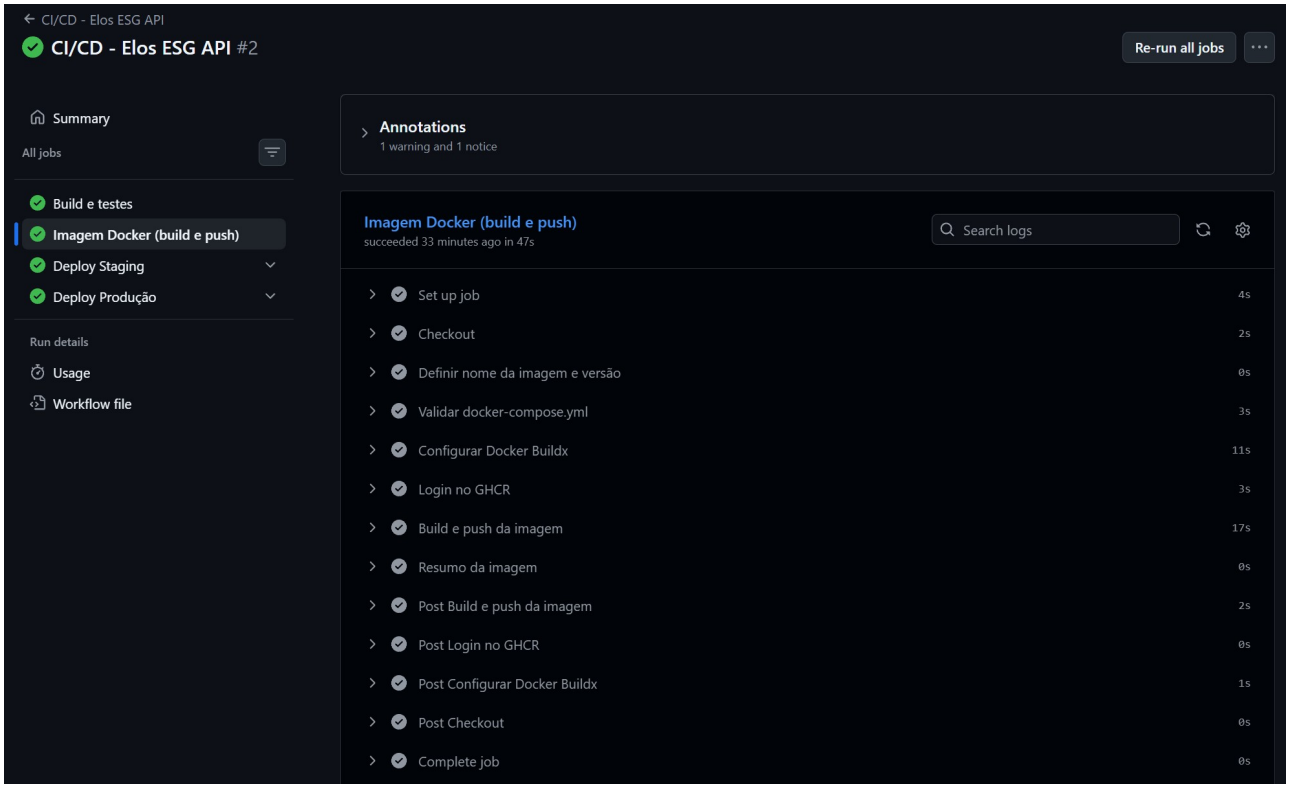
Execução #2 do workflow com os 4 jobs concluídos com sucesso.

Figura 2. Build e testes automatizados



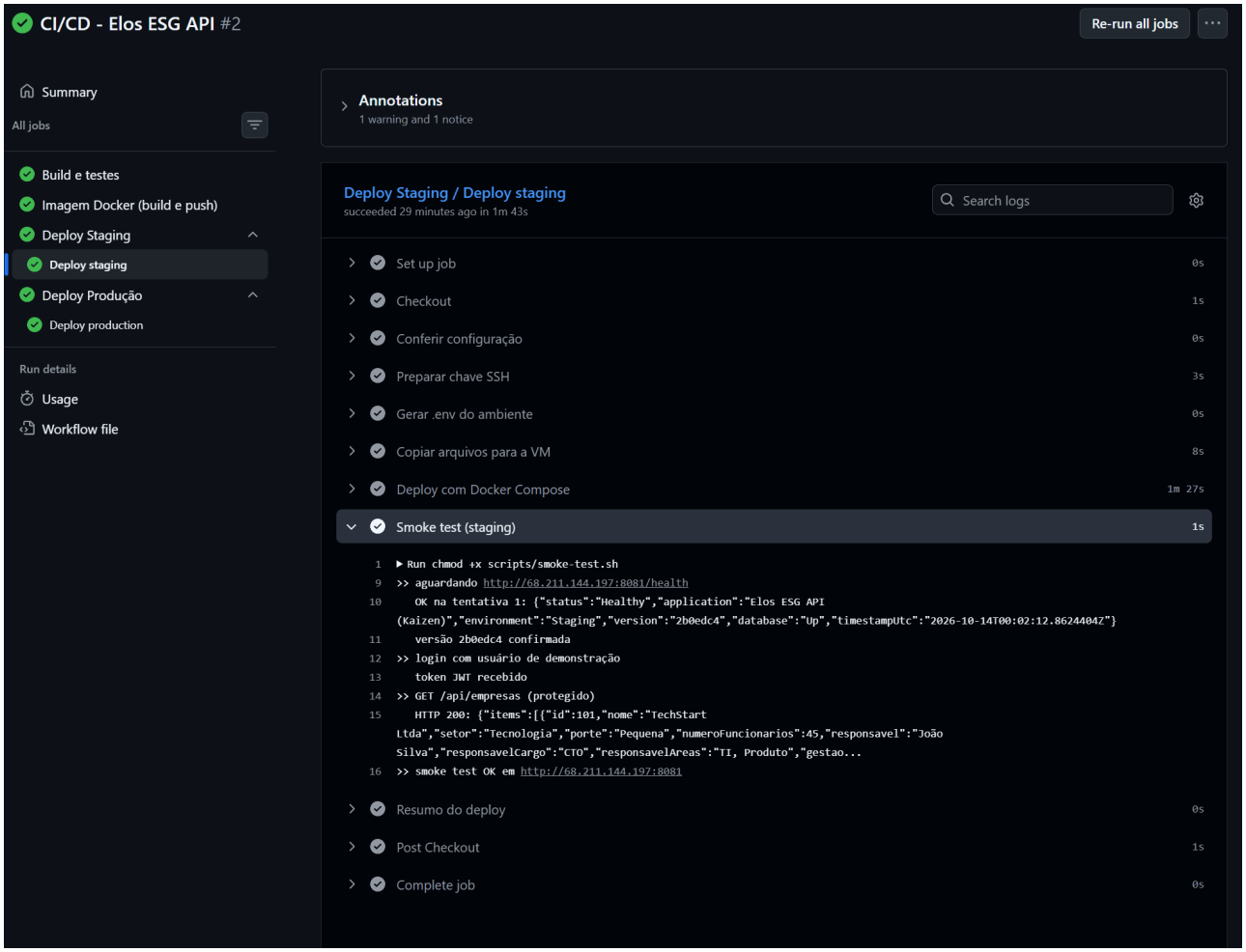
Passo de testes do job Build e testes: 26 testes executados e 26 aprovados.

Figura 3. Imagem publicada no GHCR



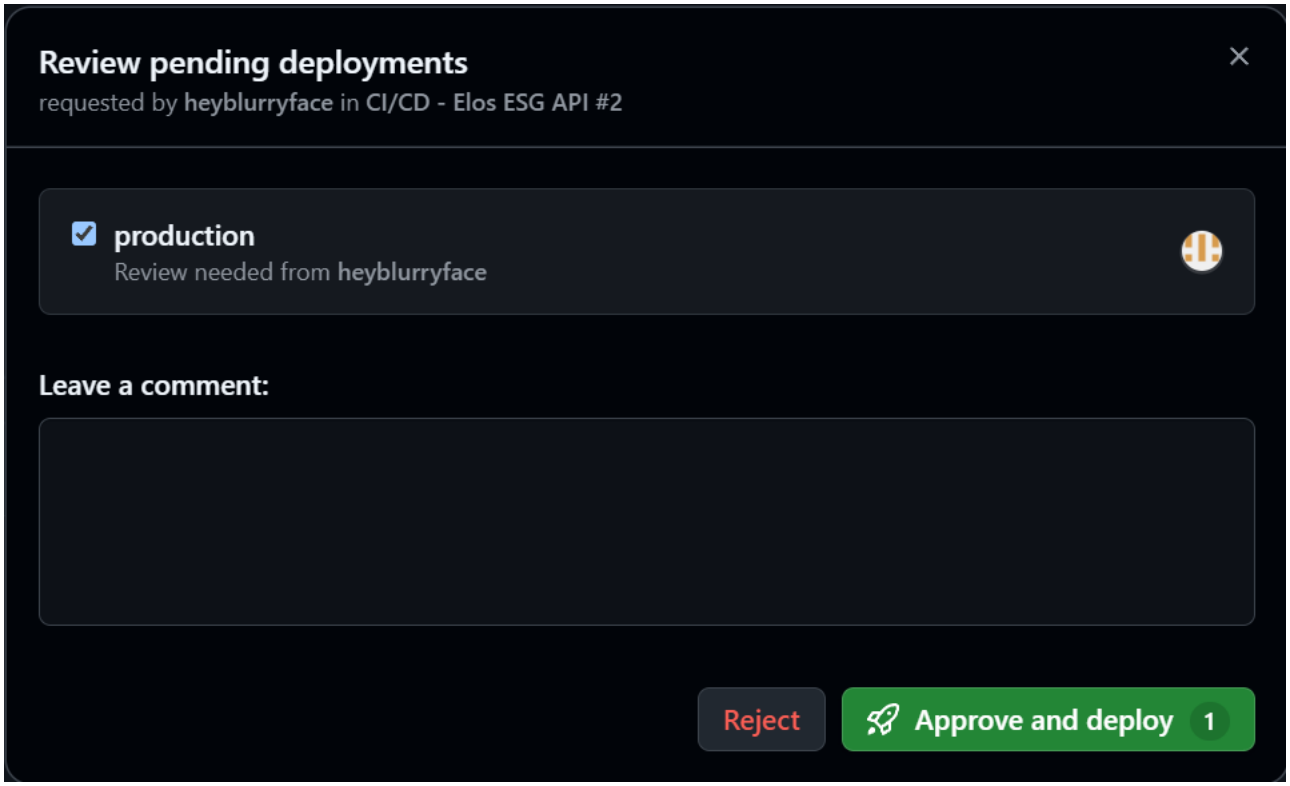
Etapas do job de imagem: validação do compose, Buildx, login no GHCR e build/push.

Figura 4. Deploy em staging



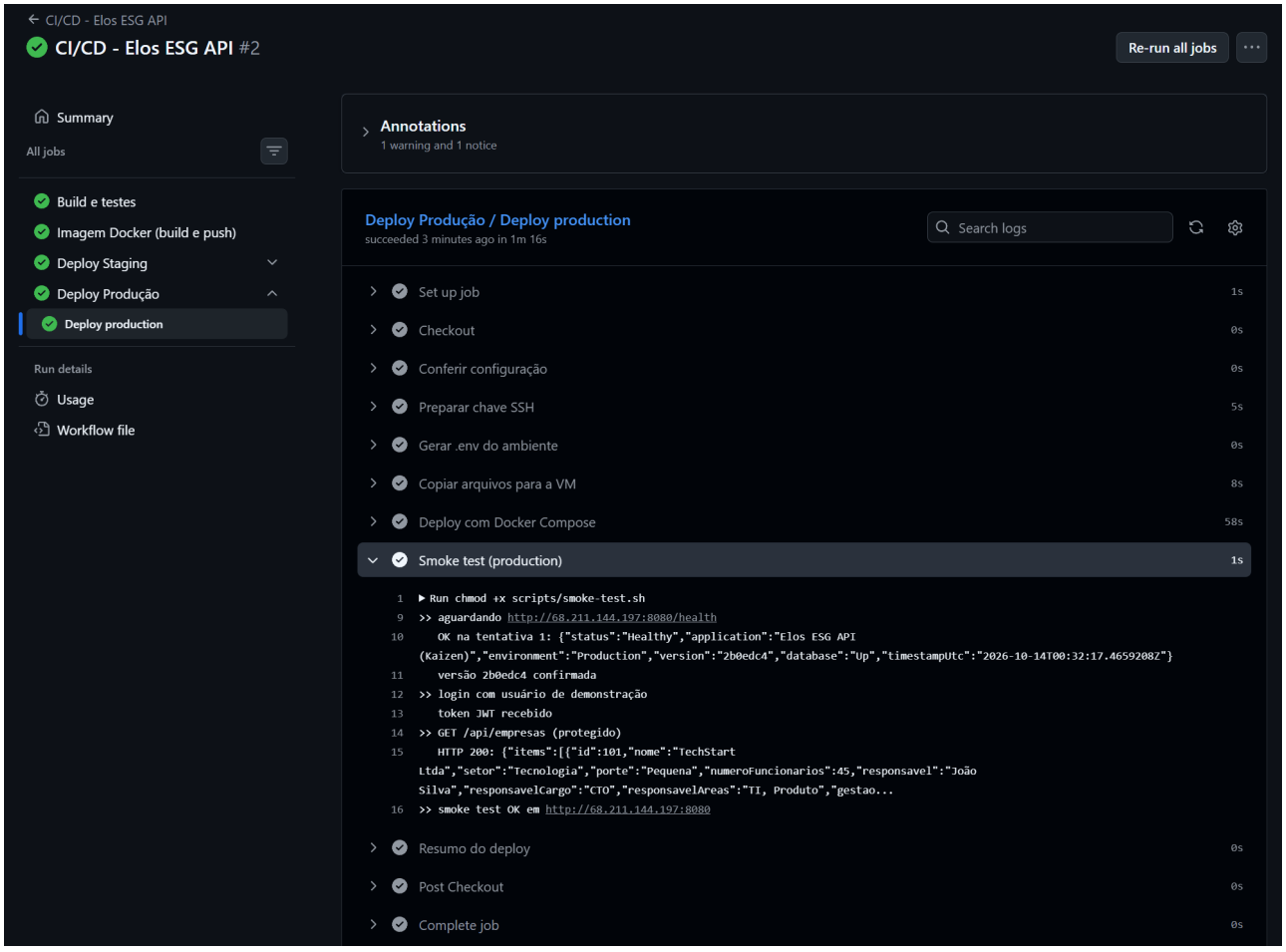
Smoke test em staging: health com a versão 2b0edc4, login JWT e GET /api/empresas com HTTP 200.

Figura 5. Aprovação manual de produção



Aprovação manual exigida pelo environment production antes do deploy.

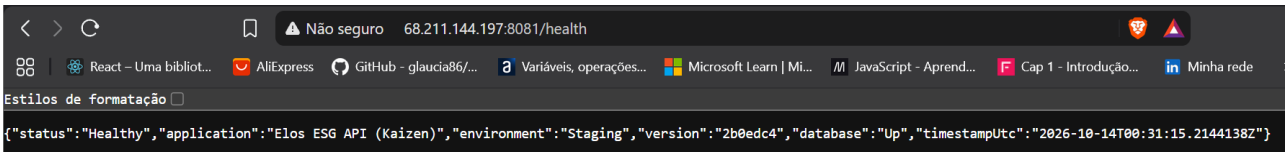
Figura 6. Deploy em produção



Smoke test em produção com a mesma versão 2b0edc4 promovida de staging.

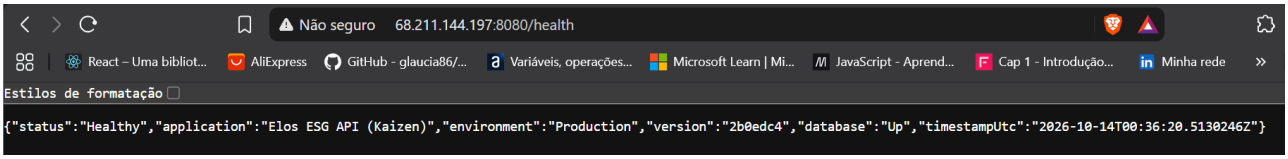
5. Evidências dos ambientes

Figura 7. Staging funcionando



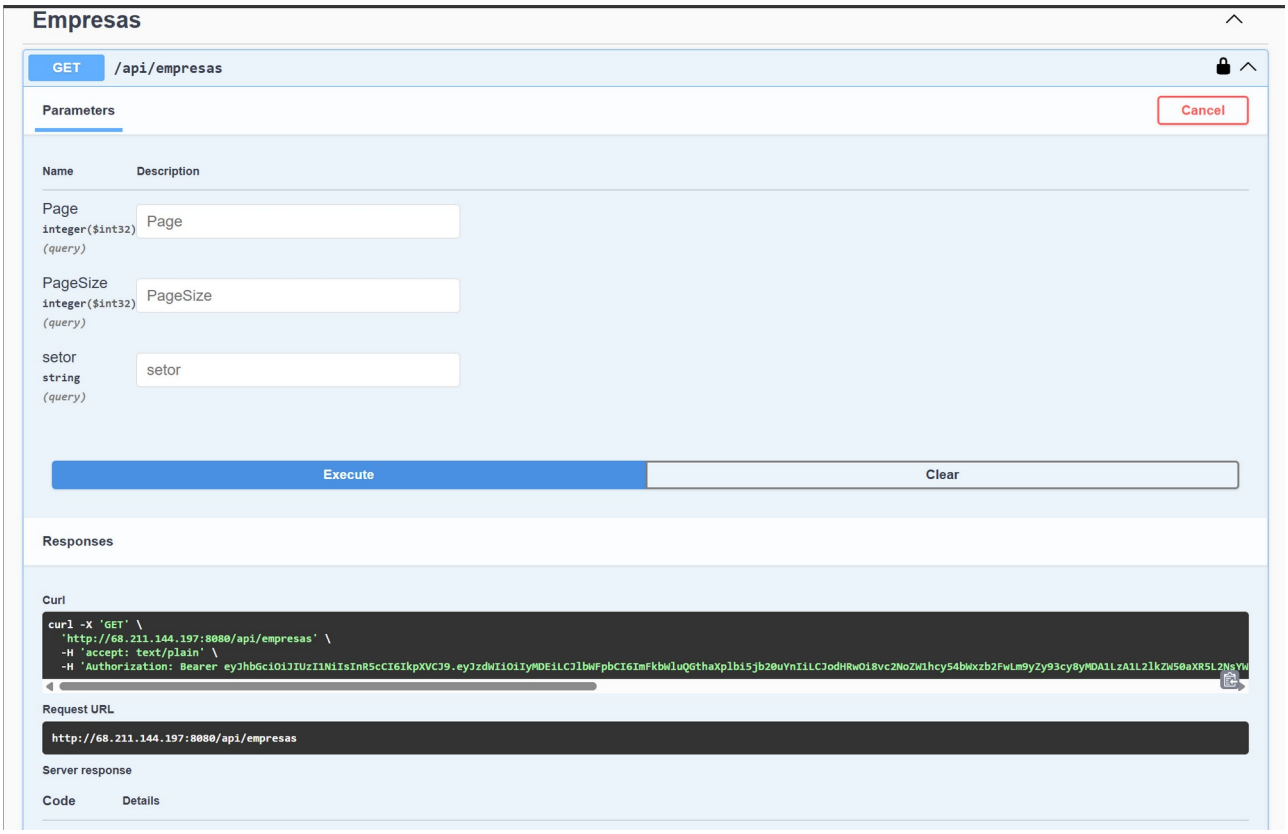
http://68.211.144.197:8081/health respondendo com environment Staging e banco Up.

Figura 8. Produção funcionando



http://68.211.144.197:8080/health respondendo com environment Production e banco Up.

Figura 9. Swagger da produção



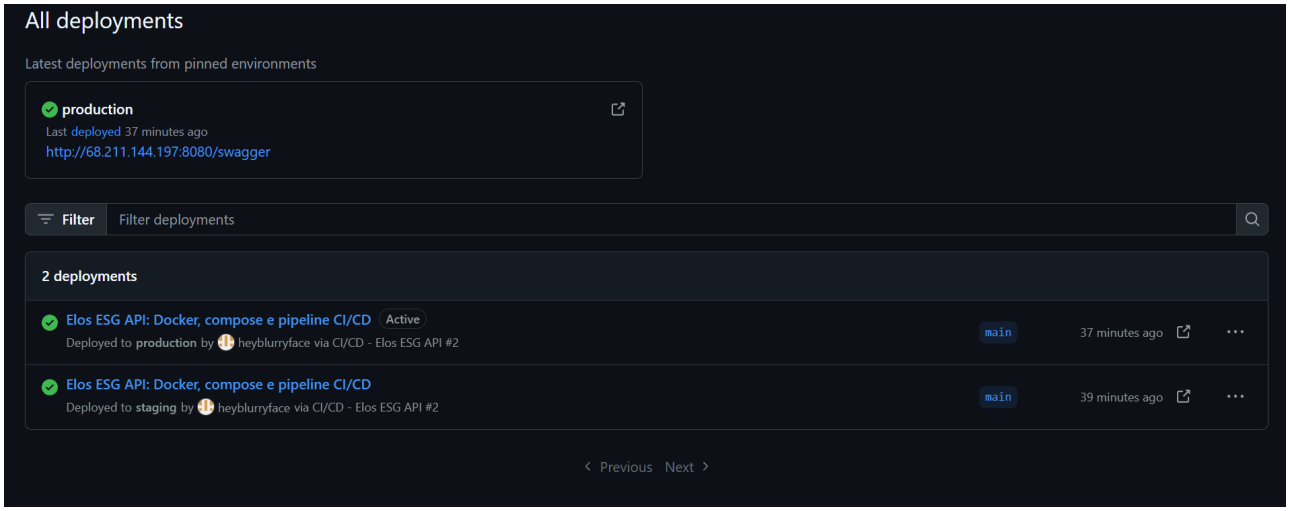
Swagger de produção: GET /api/empresas com token JWT retornando HTTP 200.

Figura 10. Containers na VM



docker ps na VM: API e Oracle de cada ambiente rodando e healthy.

Figura 11. Environments no GitHub



Deployments do GitHub com os ambientes staging e production.

6. Desafios encontrados e soluções

Desafio	Como foi resolvido
Regiões bloqueadas na assinatura Azure for Students	Na primeira tentativa a criação da VM em East US falhou com RequestDisallowedByAzure. A assinatura da faculdade tem uma policy que só libera algumas regiões. Consultei a policy com az policy assignment list e a lista permitida era Belgium Central, Italy North, Chile Central, Mexico Central e South Africa North. Usei Chile Central, a mais próxima do Brasil.
Tamanho de VM sem capacidade	O tamanho planejado (Standard_B2ms) retornou SkuNotAvailable no Chile e no México. Fiz um loop no Cloud Shell testando outros tamanhos com memória suficiente para dois Oracle e o Standard_B2s_v2 (2 vCPU, 8 GB) funcionou de primeira no Chile Central.
Hospedar o Oracle na nuvem	O banco do projeto é Oracle, que não roda em serviços gerenciados como o App Service. A saída foi usar uma VM Linux com Docker e o mesmo docker-compose.yml do ambiente local, o que também deixou os ambientes iguais ao local.
Staging e produção na mesma VM sem misturar	Cada ambiente roda como um projeto separado do Compose (-p elos-staging e -p elos-production). Assim cada um tem containers, rede, volume e senhas próprios, só a porta publicada muda.
Primeira subida do Oracle é lenta	Na primeira execução o Oracle leva alguns minutos. Foram usados healthcheck no banco, depends_on com service_healthy, retry da migration na API, docker compose up --wait com timeout de 15 minutos e smoke test com novas tentativas.
Saber se o deploy trocou mesmo a versão	Criei o endpoint /health devolvendo o ambiente e o SHA do commit (APP_VERSION). O smoke test compara essa versão com a do pipeline e falha se não bater.
Testes no CI sem banco Oracle	Os testes de integração usam WebApplicationFactory com EF Core InMemory e um handler de autenticação de teste, então o job de CI não depende de banco nenhum.
Manter segredos fora do repositório	Senhas e chave JWT ficam nos secrets de cada GitHub Environment. O .env é gerado no deploy, vai para a VM com permissão 600 e o token do GHCR é passado ao script pela entrada padrão.

Desafio	Como foi resolvido
Nome da imagem no GHCR	O registry só aceita nomes em minúsculo e o nome do repositório pode ter maiúsculas. O pipeline converte com ${GITHUB_REPOSITORY,,} antes de montar a tag.
Aprovação manual de produção	Required reviewers em environments só é liberado de graça em repositório público, então o repositório foi deixado público. O script github-setup.sh já cadastra o aprovador.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑️
Dockerfile funcional	☑️
docker-compose.yml ou arquivos Kubernetes	☑️
Pipeline com etapas de build, teste e deploy	☑️
README.md com instruções e prints	☑️
Documentação técnica com evidências (PDF ou PPT)	☑️
Deploy realizado nos ambientes staging e produção	☑️